Detecting Order-Dependent Flaky Tests

Anonymous Author(s)

I. Introduction

Flaky tests pass and fail on the same version of the code. A developer who sees such a failure cannot tell whether the change under review broke the program or whether the test fails at random. As a result, many teams rerun every failed test several times before they look at the failure, which wastes the time of the build servers and delays the review of each change. In this paper we study the tests that depend on the order in which the build runs them. Such a test passes when it runs alone and fails when another test leaves a shared state behind, such as a static field, a file in a temporary folder, or a row in a database.

We present a detector that reruns each test in a set of shuffled orders and records the outcome of every run. The detector labels a test as stable when all orders give the same outcome and as flaky when two orders give different outcomes. It labels the remaining tests as unclear when the build stops before the detector has tried enough orders. The detector runs as a plug-in of the build tool and reads the reports that the test runner writes for each run. It needs no change to the tests and no change to the code of the project.

II. Approach

For each test the detector first runs the test alone and then runs the test after each of the other tests of the same class. It keeps the orders that change the outcome and reports the pair of tests that leaves the shared state. The developer can then fix the test that pollutes the state or the test that depends on the state. In addition, the detector stores the orders that it has already tried, which allows a later build to continue where the last build stopped. The order of the remaining tests follows the time that each test took in the previous build, since short tests give more orders for the same budget of time.

The detector also records the stack trace of every failure. Two failures with the same trace in the same test count as one failure, because a test that fails for the same reason in many orders points to the same shared state. The report lists each flaky test with the orders that make it fail, the test that runs before it in these orders, and the line of the assertion that fails.

III. Evaluation

We run the detector on the tests of twelve open source projects written in Java. The projects range from small libraries with a few hundred tests to large frameworks with more than ten thousand tests. For each project we take the latest release and run the full test suite in the default order to obtain the reference outcome of every test. We then run the detector with a budget of one hour for each project and compare its labels with the labels of a manual inspection of the failures.

The detector finds flaky tests in nine of the twelve projects. Most of these tests share a static field with another test of the same class, and a few of them write a file that a later test reads. The manual inspection confirms the label of each flaky test that the detector reports. The detector misses the tests that fail only when the build runs the tests in parallel, since it runs every test in a single thread. We discuss this limit in the threats to the validity of the study and describe how a later version of the detector can run the tests in parallel.

IV. Related Work

Earlier work on flaky tests classifies the causes of flakiness and finds that the order of the tests, asynchronous calls, and concurrency account for most of the flaky tests in open source projects. Other tools rerun the tests in random orders or search for the pairs of tests that pollute a shared state. Our detector follows the same idea and adds the reuse of the orders across builds, which reduces the time that each build spends on the search.

V. Conclusion

The detector finds tests that depend on the order of the test suite at a small cost for each build. The reports name the test that leaves the shared state behind and the test that fails because of it. We plan to extend the detector to tests that run in parallel and to tests that depend on the time of the day or on the network.

VI. Introduction

Flaky tests pass and fail on the same version of the code. A developer who sees such a failure cannot tell whether the change under review broke the program or whether the test fails at random. As a result, many teams rerun every failed test several times before they look at the failure, which wastes the time of the build servers and delays the review of each change. In this paper we study the tests that depend on the order in which the build runs them. Such a test passes when it runs alone and fails when another test leaves a shared state behind, such as a static field, a file in a temporary folder, or a row in a database.

We present a detector that reruns each test in a set of shuffled orders and records the outcome of every run. The detector labels a test as stable when all orders give the same outcome and as flaky when two orders give different outcomes. It labels the remaining tests as unclear when the build stops before the detector has tried enough orders. The detector runs as a plug-in of the build tool and reads the reports that the test runner writes for each run. It needs no change to the tests and no change to the code of the project.

VII. Approach

For each test the detector first runs the test alone and then runs the test after each of the other tests of the same class. It keeps the orders that change the outcome and reports the pair of tests that leaves the shared state. The developer can then fix the test that pollutes the state or the test that depends on the state. In addition, the detector stores the orders that it has already tried, which allows a later build to continue where the last build stopped. The order of the remaining tests follows the time that each test took in the previous build, since short tests give more orders for the same budget of time.

The detector also records the stack trace of every failure. Two failures with the same trace in the same test count as one failure, because a test that fails for the same reason in many orders points to the same shared state. The report lists each flaky test with the orders that make it fail, the test that runs before it in these orders, and the line of the assertion that fails.

VIII. Evaluation

We run the detector on the tests of twelve open source projects written in Java. The projects range from small libraries with a few hundred tests to large frameworks with more than ten thousand tests. For each project we take the latest release and run the full test suite in the default order to obtain the reference outcome of every test. We then run the detector with a budget of one hour for each project and compare its labels with the labels of a manual inspection of the failures.

The detector finds flaky tests in nine of the twelve projects. Most of these tests share a static field with another test of the same class, and a few of them write a file that a later test reads. The manual inspection confirms the label of each flaky test that the detector reports. The detector misses the tests that fail only when the build runs the tests in parallel, since it runs every test in a single thread. We discuss this limit in the threats to the validity of the study and describe how a later version of the detector can run the tests in parallel.

IX. Related Work

Earlier work on flaky tests classifies the causes of flakiness and finds that the order of the tests, asynchronous calls, and concurrency account for most of the flaky tests in open source projects. Other tools rerun the tests in random orders or search for the pairs of tests that pollute a shared state. Our detector follows the same idea and adds the reuse of the orders across builds, which reduces the time that each build spends on the search.

X. Conclusion

The detector finds tests that depend on the order of the test suite at a small cost for each build. The reports name the test that leaves the shared state behind and the test that fails because of it. We plan to extend the detector to tests that run in parallel and to tests that depend on the time of the day or on the network.